EE 66 Signals, Dynamics, and Information
Fall 2026 Note 6

6.1 Matrices

This note walks through some background material in important matrices, matrix multiplication, and matrix decompositions to aid in your study of signals and systems.

6.2 Important Matrices

Definition 6.1 (Zero Matrix): Similar to a zero vector, a zero matrix is a matrix with all components equal to zero. It is usually just represented as 0, where its size is implied from context. Here is an example of a 2×2 zero matrix:

$$\mathbf{0}_{2\times 2} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Definition 6.2 (Identity Matrix): An identity matrix, often written as $\mathbf{I}$, is a square matrix whose diagonal elements are 1 and whose off-diagonal elements are all 0. Here is an example of the 3×3 identity matrix:

$$\mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Note that the column vectors (and the transpose of the row vectors) of an $n \times n$ identity matrix are the unit vectors in $\mathbb{R}^n$. The identity matrix is useful because multiplying it with a vector $\mathbf{x}$ will leave the vector unchanged - in other words, $\mathbf{Ix} = \mathbf{x}$.

In fact, we will see that multiplying a square matrix by an identity matrix of the same size will yield the same initial matrix: $\mathbf{AI} = \mathbf{IA} = \mathbf{A}$

Definition 6.3 (Symmetric Matrix): A symmetric matrix is a square matrix $\mathbf{A}$ such that $\mathbf{A} = \mathbf{A}^\top$, which means that $\mathbf{A}_{ij} = \mathbf{A}_{ji}$ for all i and j. An example of this is the identity matrix, $\mathbf{I}$, which has 1s along its diagonal and 0s everywhere else.

Symmetric matrices have three useful properties:

1. All eigenvalues of a symmetric matrix are real

UCB EE 66, Fall 2026, Note 6, All Rights Reserved. This may not be publicly shared without explicit permission.

2. Eigenvectors corresponding to distinct eigenvalues are orthogonal
3. Symmetric matrices are always diagonalizable

This will come into play when you learn about SVD, a matrix decomposition that leverages symmetric matrix properties.

Definition 6.4 (Skew-Symmetric Matrix): A skew-symmetric matrix is a square matrix $\mathbf{A}$ such that $\mathbf{A} = -\mathbf{A}^\top$, which means that $\mathbf{A}_{ij} = -\mathbf{A}_{ji}$ for all i and j. Below is an example of a skew-symmetric matrix:

$$\mathbf{A} = \begin{bmatrix} 0 & 2 & 3 \\ -2 & 0 & -1 \\ -3 & 1 & 0 \end{bmatrix}$$

Some useful properties are that for any real skew symmetric matrix $\mathbf{A}$, the real eigenvalue will be zero, the nonzero eigenvalues will be non-real, and $\mathbf{A} + \mathbf{I}$ will be invertible.

Definition 6.5 (Toeplitz Matrix): A Toeplitz matrix is a matrix $\mathbf{A}$ such that all elements along each diagonal are identical, meaning $\mathbf{A}_{ij} = \mathbf{A}_{i+1,j+1}$. An element $\mathbf{A}_{ij}$ belongs to the diagonal denoted by a_{i-j}.

$$\mathbf{A} = \begin{bmatrix} a_0 & a_{-1} & a_{-2} \\ a_1 & a_0 & a_{-1} \\ a_2 & a_1 & a_0 \end{bmatrix}$$

A Toeplitz matrix can be rectangular, and does not need to be square. For example, the following is a 3×4 Toeplitz matrix:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 1 & 2 & 3 \\ 6 & 5 & 1 & 2 \end{bmatrix}$$

6.3 Review: Views of Matrix Multiplication

Matrix multiplication shows up throughout this note, from eigendecomposition to the state transition matrices behind Page Rank. In this section we look at several equivalent ways to view matrix-vector and matrix-matrix products. If you already feel confident with the different views of matrix-matrix multiplication, feel free to skip ahead to Section 6.4. Throughout, we color the entries so that the pieces paired together in each product share a color.

6.3.1 Reading Matrix Dimensions

A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ has m rows and n columns: the dimensions are always read as rows first, then columns. The entry a_{kl} sits in row k and column l. For example,

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \in \mathbb{R}^{2 \times 3}$$

has 2 rows and 3 columns, and $a_{23} = 6$. A vector $\mathbf{x} \in \mathbb{R}^n$ is a matrix with a single column, so it is $n \times 1$, and its transpose $\mathbf{x}^\top$ is $1 \times n$.

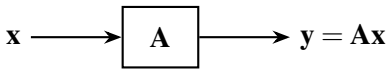
A product is defined only when the number of columns of the left matrix equals the number of rows of the right matrix (the *inner* dimensions match). The result takes the *outer* dimensions:

$$\underbrace{\mathbf{y}}_{m \times 1} = \underbrace{\mathbf{A}}_{m \times n} \underbrace{\mathbf{x}}_{n \times 1}, \qquad \underbrace{\mathbf{C}}_{m \times n} = \underbrace{\mathbf{A}}_{m \times p} \underbrace{\mathbf{B}}_{p \times n}$$

So a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ takes inputs in $\mathbb{R}^n$ (one entry per column) and produces outputs in $\mathbb{R}^m$ (one entry per row). In the running example below, a 2×3 matrix times a 3×2 matrix gives a 2×2 matrix.

6.3.2 Matrix-Vector Multiplication

A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ takes an input $\mathbf{x} \in \mathbb{R}^n$ and produces an output $\mathbf{y} = \mathbf{A}\mathbf{x} \in \mathbb{R}^m$:



As a running example,

$$\mathbf{y} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1x_1 + 2x_2 + 3x_3 \\ 4x_1 + 5x_2 + 6x_3 \end{bmatrix}$$

6.3.2.1 Dot Product View

Write $\mathbf{A}$ by its rows. In the example,

$$\mathbf{y} = \begin{bmatrix} \longleftarrow & \boldsymbol{\alpha}_1^\top & \longrightarrow \\ \longleftarrow & \boldsymbol{\alpha}_2^\top & \longrightarrow \end{bmatrix} \mathbf{x} = \begin{bmatrix} \boldsymbol{\alpha}_1^\top \mathbf{x} \\ \boldsymbol{\alpha}_2^\top \mathbf{x} \end{bmatrix} = \begin{bmatrix} \langle \boldsymbol{\alpha}_1, \mathbf{x} \rangle \\ \langle \boldsymbol{\alpha}_2, \mathbf{x} \rangle \end{bmatrix}$$

More generally, if $\mathbf{A}$ has rows $\boldsymbol{\alpha}_1^\top, \ldots, \boldsymbol{\alpha}_m^\top$, then

$$\mathbf{y} = \mathbf{A}\mathbf{x} = \begin{bmatrix} \longleftarrow & \boldsymbol{\alpha}_1^\top & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\alpha}_k^\top & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\alpha}_m^\top & \longrightarrow \end{bmatrix} \mathbf{x} = \begin{bmatrix} \boldsymbol{\alpha}_1^\top \mathbf{x} \\ \vdots \\ \boldsymbol{\alpha}_k^\top \mathbf{x} \\ \vdots \\ \boldsymbol{\alpha}_m^\top \mathbf{x} \end{bmatrix}, \qquad \boxed{y_k = \boldsymbol{\alpha}_k^\top \mathbf{x}, \quad k = 1, \ldots, m}$$

Each entry of $\mathbf{y}$ is the dot product of a row of $\mathbf{A}$ with $\mathbf{x}$.

6.3.2.2 Column Linear Combination View

Now split the running example by columns instead:

$$\mathbf{y} = \begin{bmatrix} 1x_1 + 2x_2 + 3x_3 \\ 4x_1 + 5x_2 + 6x_3 \end{bmatrix} = \begin{bmatrix} 1x_1 \\ 4x_1 \end{bmatrix} + \begin{bmatrix} 2x_2 \\ 5x_2 \end{bmatrix} + \begin{bmatrix} 3x_3 \\ 6x_3 \end{bmatrix}$$
$$= x_1 \underbrace{\begin{bmatrix} 1 \\ 4 \end{bmatrix}}_{\mathbf{a}_1} + x_2 \underbrace{\begin{bmatrix} 2 \\ 5 \end{bmatrix}}_{\mathbf{a}_2} + x_3 \underbrace{\begin{bmatrix} 3 \\ 6 \end{bmatrix}}_{\mathbf{a}_3} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + x_3\mathbf{a}_3$$

More generally, if $\mathbf{A} = \begin{bmatrix} \mathbf{a}_1 & \cdots & \mathbf{a}_n \end{bmatrix}$ with columns $\mathbf{a}_l \in \mathbb{R}^m$, then

$$\boxed{\mathbf{A}\mathbf{x} = x_1\mathbf{a}_1 + \cdots + x_l\mathbf{a}_l + \cdots + x_n\mathbf{a}_n = \sum_{l=1}^{n} x_l\mathbf{a}_l}$$

That is, $\mathbf{A}\mathbf{x}$ is a linear combination of the columns of $\mathbf{A}$, weighted by the entries of $\mathbf{x}$.

The two views give two different readings of the same equation. Suppose $\mathbf{A}\mathbf{x} = \mathbf{0}$ for some $\mathbf{x} \neq \mathbf{0}$.

1. In the column view,
$$\mathbf{A}\mathbf{x} = \sum_{l=1}^{n} x_l\mathbf{a}_l = \mathbf{0}$$
is a nontrivial linear combination of $\mathbf{a}_1, \ldots, \mathbf{a}_n$ that produces the zero vector, so the columns of $\mathbf{A}$ are linearly dependent.
2. In the dot product view, $\boldsymbol{\alpha}_k^\top \mathbf{x} = 0$ for every k, so $\mathbf{x}$ is orthogonal to every row $\boldsymbol{\alpha}_1, \ldots, \boldsymbol{\alpha}_m$.

6.3.3 Matrix-Matrix Multiplication

Applying $\mathbf{B} \in \mathbb{R}^{p \times n}$ to $\mathbf{x} \in \mathbb{R}^n$ and then applying $\mathbf{A} \in \mathbb{R}^{m \times p}$ to the result gives $\mathbf{A}(\mathbf{B}\mathbf{x}) = \mathbf{C}\mathbf{x}$, where $\mathbf{C} = \mathbf{A}\mathbf{B} \in \mathbb{R}^{m \times n}$. As a running example,

$$\mathbf{A}\mathbf{B} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 0 & 3 \\ 1 & 4 \\ 2 & 5 \end{bmatrix} = \begin{bmatrix} 1\cdot 0 + 2\cdot 1 + 3\cdot 2 & 1\cdot 3 + 2\cdot 4 + 3\cdot 5 \\ 4\cdot 0 + 5\cdot 1 + 6\cdot 2 & 4\cdot 3 + 5\cdot 4 + 6\cdot 5 \end{bmatrix} = \begin{bmatrix} 8 & 26 \\ 17 & 62 \end{bmatrix} = \mathbf{C}$$

Each term pairs an entry from the r-th column of $\mathbf{A}$ with an entry from the r-th row of $\mathbf{B}$, so those share a color: orange for $r = 1$, pink for $r = 2$, and green for $r = 3$.

To describe the views below, we write $\mathbf{A}$ and $\mathbf{B}$ by their columns and by their rows:

$$\mathbf{A} = \begin{bmatrix} \uparrow & & \uparrow & & \uparrow \\ \mathbf{a}_1 & \cdots & \mathbf{a}_l & \cdots & \mathbf{a}_p \\ \downarrow & & \downarrow & & \downarrow \end{bmatrix} = \begin{bmatrix} \leftarrow & \boldsymbol{\alpha}_1^\top & \rightarrow \\ & \vdots & \\ \leftarrow & \boldsymbol{\alpha}_k^\top & \rightarrow \\ & \vdots & \\ \leftarrow & \boldsymbol{\alpha}_m^\top & \rightarrow \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} \uparrow & & \uparrow & & \uparrow \\ \mathbf{b}_1 & \cdots & \mathbf{b}_l & \cdots & \mathbf{b}_n \\ \downarrow & & \downarrow & & \downarrow \end{bmatrix} = \begin{bmatrix} \leftarrow & \boldsymbol{\beta}_1^\top & \rightarrow \\ & \vdots & \\ \leftarrow & \boldsymbol{\beta}_k^\top & \rightarrow \\ & \vdots & \\ \leftarrow & \boldsymbol{\beta}_p^\top & \rightarrow \end{bmatrix}$$

We write a_{kr} and b_{rl} for the individual entries of $\mathbf{A}$ and $\mathbf{B}$, and c_{kl} for the entries of $\mathbf{C}$.

6.3.3.1 (I) Inner Product View

$$\mathbf{C} = \mathbf{AB} = \begin{bmatrix} \leftarrow & \boldsymbol{\alpha}_1^\top & \rightarrow \\ & \vdots & \\ \leftarrow & \boldsymbol{\alpha}_k^\top & \rightarrow \\ & \vdots & \\ \leftarrow & \boldsymbol{\alpha}_m^\top & \rightarrow \end{bmatrix} \begin{bmatrix} \uparrow & & \uparrow & & \uparrow \\ \mathbf{b}_1 & \cdots & \mathbf{b}_l & \cdots & \mathbf{b}_n \\ \downarrow & & \downarrow & & \downarrow \end{bmatrix} = \begin{bmatrix} \boldsymbol{\alpha}_1^\top \mathbf{b}_1 & \cdots & \boldsymbol{\alpha}_1^\top \mathbf{b}_l & \cdots & \boldsymbol{\alpha}_1^\top \mathbf{b}_n \\ \vdots & & \vdots & & \vdots \\ \boldsymbol{\alpha}_k^\top \mathbf{b}_1 & \cdots & \boldsymbol{\alpha}_k^\top \mathbf{b}_l & \cdots & \boldsymbol{\alpha}_k^\top \mathbf{b}_n \\ \vdots & & \vdots & & \vdots \\ \boldsymbol{\alpha}_m^\top \mathbf{b}_1 & \cdots & \boldsymbol{\alpha}_m^\top \mathbf{b}_l & \cdots & \boldsymbol{\alpha}_m^\top \mathbf{b}_n \end{bmatrix}$$

Each entry of $\mathbf{C}$ is the inner product of a row of $\mathbf{A}$ with a column of $\mathbf{B}$:

$$\boxed{c_{kl} = \boldsymbol{\alpha}_k^\top \mathbf{b}_l}, \qquad k \in \{1, \ldots, m\}, \quad l \in \{1, \ldots, n\}$$

In the example, $c_{11} = \boldsymbol{\alpha}_1^\top \mathbf{b}_1 = 1 \cdot 0 + 2 \cdot 1 + 3 \cdot 2 = 8$.

6.3.3.2 (II) Columns of $\mathbf{C}$ as Linear Combinations of the Columns of $\mathbf{A}$

Each column of $\mathbf{C}$ is $\mathbf{A}$ applied to the matching column of $\mathbf{B}$: $\mathbf{c}_1 = \mathbf{Ab}_1, \ldots, \mathbf{c}_l = \mathbf{Ab}_l, \ldots, \mathbf{c}_n = \mathbf{Ab}_n$. Stacking these side by side,

$$\underbrace{\begin{bmatrix} \uparrow & & \uparrow & & \uparrow \\ \mathbf{c}_1 & \cdots & \mathbf{c}_l & \cdots & \mathbf{c}_n \\ \downarrow & & \downarrow & & \downarrow \end{bmatrix}}_{\mathbf{C}} = \begin{bmatrix} \uparrow & & \uparrow & & \uparrow \\ \mathbf{Ab}_1 & \cdots & \mathbf{Ab}_l & \cdots & \mathbf{Ab}_n \\ \downarrow & & \downarrow & & \downarrow \end{bmatrix} = \mathbf{A} \underbrace{\begin{bmatrix} \uparrow & & \uparrow & & \uparrow \\ \mathbf{b}_1 & \cdots & \mathbf{b}_l & \cdots & \mathbf{b}_n \\ \downarrow & & \downarrow & & \downarrow \end{bmatrix}}_{\mathbf{B}}$$

In the example,

$$\mathbf{AB} = \Bigg[\underbrace{0\begin{bmatrix}1\\4\end{bmatrix} + 1\begin{bmatrix}2\\5\end{bmatrix} + 2\begin{bmatrix}3\\6\end{bmatrix}}_{\mathbf{c}_1 = \mathbf{A}\mathbf{b}_1} \quad \underbrace{3\begin{bmatrix}1\\4\end{bmatrix} + 4\begin{bmatrix}2\\5\end{bmatrix} + 5\begin{bmatrix}3\\6\end{bmatrix}}_{\mathbf{c}_2 = \mathbf{A}\mathbf{b}_2}\Bigg] = \begin{bmatrix}8 & 26\\17 & 62\end{bmatrix} = \mathbf{C}$$

6.3.3.3 (III) Rows of $\mathbf{C}$ as Linear Combinations of the Rows of $\mathbf{B}$

Let $\boldsymbol{\gamma}_k^\top$ denote the k-th row of $\mathbf{C}$. Each row of $\mathbf{C}$ is the matching row of $\mathbf{A}$ applied to $\mathbf{B}$:

$$\mathbf{C} = \begin{bmatrix} \longleftarrow & \boldsymbol{\gamma}_1^\top & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\gamma}_k^\top & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\gamma}_m^\top & \longrightarrow \end{bmatrix} = \begin{bmatrix} \longleftarrow & \boldsymbol{\alpha}_1^\top & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\alpha}_k^\top & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\alpha}_m^\top & \longrightarrow \end{bmatrix} \mathbf{B} = \begin{bmatrix} \longleftarrow & \boldsymbol{\alpha}_1^\top \mathbf{B} & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\alpha}_k^\top \mathbf{B} & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\alpha}_m^\top \mathbf{B} & \longrightarrow \end{bmatrix}$$

In the example,

$$\mathbf{AB} = \begin{bmatrix} 1\begin{bmatrix}0 & 3\end{bmatrix} + 2\begin{bmatrix}1 & 4\end{bmatrix} + 3\begin{bmatrix}2 & 5\end{bmatrix} \\ 4\begin{bmatrix}0 & 3\end{bmatrix} + 5\begin{bmatrix}1 & 4\end{bmatrix} + 6\begin{bmatrix}2 & 5\end{bmatrix} \end{bmatrix} = \begin{bmatrix}8 & 26\\17 & 62\end{bmatrix} = \mathbf{C}$$

6.3.3.4 (IV) $\mathbf{C}$ as a Sum of Outer Products of the Columns of $\mathbf{A}$ and the Rows of $\mathbf{B}$

Write $\mathbf{A}$ by its columns and $\mathbf{B}$ by its rows:

$$\mathbf{C} = \begin{bmatrix} \uparrow & & \uparrow & & \uparrow \\ \mathbf{a}_1 & \cdots & \mathbf{a}_l & \cdots & \mathbf{a}_p \\ \downarrow & & \downarrow & & \downarrow \end{bmatrix} \begin{bmatrix} \longleftarrow & \boldsymbol{\beta}_1^\top & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\beta}_l^\top & \longrightarrow \\ & \vdots & \\ \longleftarrow & \boldsymbol{\beta}_p^\top & \longrightarrow \end{bmatrix} = \mathbf{a}_1\boldsymbol{\beta}_1^\top + \cdots + \mathbf{a}_l\boldsymbol{\beta}_l^\top + \cdots + \mathbf{a}_p\boldsymbol{\beta}_p^\top$$

That is,

$$\boxed{\mathbf{C} = \sum_{l=1}^{p} \mathbf{a}_l \boldsymbol{\beta}_l^\top}$$

where each term $\mathbf{a}_l \boldsymbol{\beta}_l^\top$, an $m \times 1$ column times a $1 \times n$ row, is an $m \times n$ matrix called an **outer product**. In the example,

$$\begin{aligned}\mathbf{AB} &= \begin{bmatrix}1\\4\end{bmatrix}\begin{bmatrix}0 & 3\end{bmatrix} + \begin{bmatrix}2\\5\end{bmatrix}\begin{bmatrix}1 & 4\end{bmatrix} + \begin{bmatrix}3\\6\end{bmatrix}\begin{bmatrix}2 & 5\end{bmatrix} \\ &= \begin{bmatrix}1\cdot 0 & 1\cdot 3\\ 4\cdot 0 & 4\cdot 3\end{bmatrix} + \begin{bmatrix}2\cdot 1 & 2\cdot 4\\ 5\cdot 1 & 5\cdot 4\end{bmatrix} + \begin{bmatrix}3\cdot 2 & 3\cdot 5\\ 6\cdot 2 & 6\cdot 5\end{bmatrix} \\ &= \begin{bmatrix}8 & 26\\ 17 & 62\end{bmatrix} = \mathbf{C}\end{aligned}$$

We will use view (II) in Section 6.4 to collect eigenvectors into a single matrix equation, and view (IV) will reappear when we study the SVD.

6.4 Eigendecomposition (Diagonalization)

In this section we will focus on a powerful matrix factorization called **eigendecomposition** or **diagonalization**. This process allows us to represent a matrix in terms of its fundamental properties: its eigenvalues and eigenvectors.

Definition 6.6 (Diagonalizable Matrix): Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be a square matrix. We say that $\mathbf{A}$ is *diagonalizable* if it can be written in the form:

$$\mathbf{A} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^{-1} \tag{1}$$

where:

- $\mathbf{V} \in \mathbb{R}^{n\times n}$ is an invertible matrix whose columns are the linearly independent eigenvectors of $\mathbf{A}$.
- $\boldsymbol{\Lambda} \in \mathbb{R}^{n\times n}$ is a diagonal matrix whose entries are the eigenvalues of $\mathbf{A}$, corresponding to the eigenvectors in $\mathbf{V}$.

Eigendecomposition is possible if and only if the matrix $\mathbf{A}$ has a full set of n linearly independent eigenvectors. A sufficient (but not necessary) condition for this is that all eigenvalues of $\mathbf{A}$ are distinct.

The derivation begins with the definition of an eigenpair $(\lambda_i, \mathbf{v}_i)$:

$$\mathbf{A}\mathbf{v}_i = \lambda_i \mathbf{v}_i \quad \text{for } i = 1, \ldots, n \tag{2}$$

Using view (II) of matrix-matrix multiplication (Section 6.3), we can express all n of these vector equations as a single matrix equation:

$$\mathbf{A}\underbrace{\begin{bmatrix}\uparrow & & \uparrow\\ \mathbf{v}_1 & \cdots & \mathbf{v}_n\\ \downarrow & & \downarrow\end{bmatrix}}_{\mathbf{V}} = \underbrace{\begin{bmatrix}\uparrow & & \uparrow\\ \lambda_1\mathbf{v}_1 & \cdots & \lambda_n\mathbf{v}_n\\ \downarrow & & \downarrow\end{bmatrix}}_{} \tag{3}$$

The right-hand side can be factored:

$$\begin{bmatrix} \uparrow & & \uparrow \\ \lambda_1 \mathbf{v}_1 & \cdots & \lambda_n \mathbf{v}_n \\ \downarrow & & \downarrow \end{bmatrix} = \underbrace{\begin{bmatrix} \uparrow & & \uparrow \\ \mathbf{v}_1 & \cdots & \mathbf{v}_n \\ \downarrow & & \downarrow \end{bmatrix}}_{\mathbf{V}} \underbrace{\begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_n \end{bmatrix}}_{\mathbf{\Lambda}} \tag{4}$$

This gives us the compact form $\mathbf{AV} = \mathbf{V\Lambda}$. Since the eigenvectors are linearly independent, $\mathbf{V}$ is invertible, and we can right-multiply by $\mathbf{V}^{-1}$ to get the eigendecomposition $\mathbf{A} = \mathbf{V\Lambda V}^{-1}$. Alternatively, left-multiplying by $\mathbf{V}^{-1}$ shows that $\mathbf{V}$ "diagonalizes" $\mathbf{A}$: $\mathbf{\Lambda} = \mathbf{V}^{-1}\mathbf{AV}$.

6.5 Powers of Diagonalizable Matrices

One of the most significant advantages of diagonalization is the ease with which we can compute matrix powers. Let's find $\mathbf{C}^2$ for a diagonalizable matrix $\mathbf{C} = \mathbf{V\Lambda V}^{-1}$:

$$\mathbf{C}^2 = (\mathbf{V\Lambda V}^{-1})(\mathbf{V\Lambda V}^{-1}) = \mathbf{V\Lambda}(\mathbf{V}^{-1}\mathbf{V})\mathbf{\Lambda V}^{-1} = \mathbf{V\Lambda I\Lambda V}^{-1} = \mathbf{V\Lambda}^2\mathbf{V}^{-1} \tag{5}$$

Extending this pattern, the t-th power of $\mathbf{C}$ is given by:

$$\mathbf{C}^t = \mathbf{V\Lambda}^t\mathbf{V}^{-1} \tag{6}$$

where $\mathbf{\Lambda}^t$ is simply the diagonal matrix of eigenvalues, with each eigenvalue raised to the power of t.

$$\mathbf{\Lambda}^t = \begin{bmatrix} \lambda_1^t & & 0 \\ & \ddots & \\ 0 & & \lambda_n^t \end{bmatrix} \tag{7}$$

6.6 Introduction to State Transition Matrices: Water Reservoirs and Pumps

To ease into state transition matrices, let's first look at a physical system of water reservoirs and pumps.

For these examples, we will have three water reservoirs, A, B, C. Let's say the initial amounts of water they respectively hold are A_0, B_0, C_0. Next, say we have a system of pumps connecting the reservoirs that move certain amounts of water between the reservoirs every day. We can represent the reservoirs as the following vector, where each element describes how much water is currently in that reservoir:

$$\begin{bmatrix} A \\ B \\ C \end{bmatrix}$$

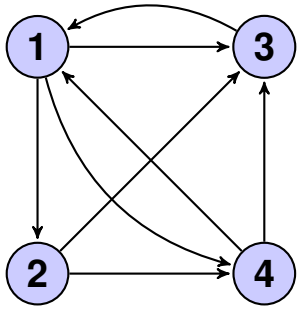


Figure 1: Assume each edge exiting each node is equally weighted. For example, page 1 has three outgoing links, so a surfer on page 1 has a $\frac{1}{3}$ chance of going to page 2, a $\frac{1}{3}$ chance of going to page 3, and a $\frac{1}{3}$ chance of going to page 4.

Then, we can represent the system of pumps as a matrix:

$$\begin{bmatrix} P_{A\leftarrow A} & P_{A\leftarrow B} & P_{A\leftarrow C} \\ P_{B\leftarrow A} & P_{B\leftarrow B} & P_{B\leftarrow C} \\ P_{C\leftarrow A} & P_{C\leftarrow B} & P_{C\leftarrow C} \end{bmatrix}$$

Each element $P_{i\leftarrow j}$ represents the fraction of water in reservoir j that goes into reservoir i the next day. The matrix acts on the vector just as the pumps act on the reservoirs, performing a transformation — multiplying a vector representing the distribution of water in one day by the pump matrix will give a vector with the distribution of water the next day. We call this matrix a **state transition matrix**. This example can also extend to matrix-matrix multiplication. Both this pumps and reservoirs example and a similar example (PageRank — how search engines can use link information to figure out which pages are important) will show further applications of linear algebra.

6.7 Page Rank: Eigenvalues and Eigenvectors in Action

Google's Page Rank algorithm uses a web crawler to estimate the popularity of webpages. We assume that the more popular a website, the more incoming links it has. We use the following setup to model the problem: suppose we have a large population of web surfers scattered at random around the internet. At each time step, all surfers randomly choose an outgoing link on their current page to arrive at a new webpage. After a large number of time steps, we expect the number of viewers on each page to reflect that page's popularity.

It's important to realize that this is a simplified model that leaves out certain characteristics of the real world: for example, people do not click links at random and people often seek out entirely new pages rather than clicking on outgoing links. Page Rank as it's used in practice, while still a simplified model, has many more intricacies than we cover here. However, even this first order approximation is a very powerful tool.

Let's look at an internet consisting of four webpages:

How can we calculate the number of viewers on each page after infinite time steps? We'll use eigenanalysis to answer this question.

Let's say we started off with 6 people on each webpage. Then one time step ticks and everyone randomly selects an outgoing link to visit. How many people do we expect to be on each webpage now?

Page 1: Page 3 only links to page 1, so all 6 people currently on 3 will move to 1. Page 4 links to pages 1 and 3, so on average about half the people on page 4 will go to page 1. There are no other incoming links, so we expect page 1 to have about $6 \times 1 + 6 \times \frac{1}{2} = 9$ people in the next time step.

Page 2: Page 1 is the only page that links to page 2. We expect a third of people from page 1 to move to page 2, so we expect page 2 to have $6 \times \frac{1}{3} = 2$ people in the next time step.

Page 3: We can arrive at page 3 from all the other pages. At page 1, the fraction of the people who move to page 3 is $\frac{1}{3}$, at page 2 the fraction of the people who move to page 3 is $\frac{1}{2}$, and at page 4 the fraction of the people who move to page 3 is $\frac{1}{2}$. Hence, we expect $6 \times \frac{1}{3} + 6 \times \frac{1}{2} + 6 \times \frac{1}{2} = 8$ people to be on page 3 in the next time step.

Page 4: We can arrive at page 4 from either page 1 or page 2. At page 1, the fraction of people who move to page 4 is $\frac{1}{3}$; at page 2, the fraction of people who move to page 4 is $\frac{1}{2}$. We expect $6 \times \frac{1}{3} + 6 \times \frac{1}{2} = 5$ people to be on page 4 in the next time step.

Notice that to find the number of people we expect to be on page i, we added up the number of people we expected would move from every other page to page i. In general, if we have k pages, $x_i[n]$ is the number of people on page i at time step n, and $p(x,y)$ is the fraction of people that will jump from page y to page x, then

$$x_i[n+1] \;=\; \sum_{j=1}^{k} x_j[n]\, p(i,j)$$

If we want to compute the expected number of viewers at time step $n+1$ for all pages simultaneously, we can use matrix notation as we did with the pump and reservoir system we looked at earlier. Define $\mathbf{x}[n] = \begin{bmatrix} x_1[n] & x_2[n] & \dots & x_k[n] \end{bmatrix}^\top$ as the vector encoding the number of viewers on each page at time step n. Then

$$\mathbf{x}[n+1] \;=\; \begin{bmatrix} x_1[n+1] \\ x_2[n+1] \\ \vdots \\ x_k[n+1] \end{bmatrix} \;=\; \begin{bmatrix} p(1,1) & p(1,2) & \dots & p(1,k) \\ p(2,1) & p(2,2) & \dots & p(2,k) \\ \vdots & \vdots & \vdots & \vdots \\ p(k,1) & p(k,2) & \dots & p(k,k) \end{bmatrix} \begin{bmatrix} x_1[n] \\ x_2[n] \\ \vdots \\ x_k[n] \end{bmatrix}$$

We call the matrix of fractions $\mathbf{P}$. Check that the entry $x_i[n+1]$ does in fact match the summation above. For our specific example, the matrix equation looks like this:

$$\begin{bmatrix} 0 & 0 & 1 & \frac{1}{2} \\ \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{3} & \frac{1}{2} & 0 & \frac{1}{2} \\ \frac{1}{3} & \frac{1}{2} & 0 & 0 \end{bmatrix} \begin{bmatrix} 6 \\ 6 \\ 6 \\ 6 \end{bmatrix} = \begin{bmatrix} 9 \\ 2 \\ 8 \\ 5 \end{bmatrix}$$

If we start with initial counts $\mathbf{x}[0]$ and want to find the expected number of viewers on each page at time step n, we compute

$$\mathbf{x}[n] = \mathbf{P}^n \mathbf{x}[0] = \begin{bmatrix} 0 & 0 & 1 & \frac{1}{2} \\ \frac{1}{3} & 0 & 0 & 0 \\ \frac{1}{3} & \frac{1}{2} & 0 & \frac{1}{2} \\ \frac{1}{3} & \frac{1}{2} & 0 & 0 \end{bmatrix}^n \mathbf{x}[0]$$

Since we only care about the relative popularity of each webpage, we can think of $\mathbf{x}[n]$ as fractions of viewers on each page (instead of the total number of viewers). We assume the viewers are initially distributed equally on all webpages, and we initialize $\mathbf{x}[0]$ as $\begin{bmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{bmatrix}^\top$. After running the simulation for many time steps, we hope $\mathbf{x}[n]$ will reflect the popularity of the webpages. This system is only useful to us if the values of $\mathbf{x}[n]$ *converge* to some stable fractions, i.e. the fraction of people on each webpage does not change at every time step.

Once our fractions exactly match these "stable fractions", running the simulation for another time step should not change the values. This means that if the fractions converge, at some point we'll reach fractions $\mathbf{x}^*$ such that

$$\mathbf{x}^* = \mathbf{P}\mathbf{x}^*$$

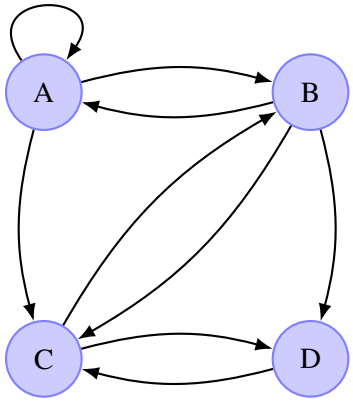


Figure 2: A simple network of four web pages (nodes) and the hyperlinks (directed edges) between them. The state vector $\mathbf{s}[n]$ would represent the probability distribution of a random surfer being on each page at time step n. The state-transition matrix $\mathbf{P}$ is constructed from these links.

In summary, we can analyze dynamic systems where the state evolves over discrete time steps using eigenanalysis. A system's state at time step n, represented by a vector $\mathbf{s}[n]$, can evolve according to the rule:

$$\mathbf{s}[n+1] = \mathbf{P}\mathbf{s}[n] \tag{8}$$

Here, $\mathbf{P}$ is the **state-transition matrix**. Starting from an initial state $\mathbf{s}[0]$, the state at time step n is:

$$\mathbf{s}[n] = \mathbf{P}^n\mathbf{s}[0] \tag{9}$$

Using the eigendecomposition of $\mathbf{P}$, we can express the state as:

$$\mathbf{s}[n] = (\mathbf{V}\mathbf{\Lambda}^n\mathbf{V}^{-1})\mathbf{s}[0] \tag{10}$$

6.8 Limiting State of the System

A key question is the long-term behavior of the system, i.e., the **limiting state** as $n \to \infty$. Let's consider the state-transition matrices used in web ranking (like PageRank). These matrices are often **column-stochastic**, meaning all their entries are non-negative and each column sums to one. A key property of any column-stochastic matrix $\mathbf{P}$ is that $\mathbf{1}^\top\mathbf{P} = \mathbf{1}$, which means $\lambda = 1$ is an eigenvalue of $\mathbf{P}^\top$. Since a matrix and its transpose share the same eigenvalues, $\mathbf{P}$ must have an eigenvalue of $\lambda_1 = 1$.

If the other eigenvalues satisfy $|\lambda_i| < 1$ for $i > 1$, then as $n \to \infty$, the terms λ_i^n will approach zero. The state evolution equation (10) simplifies to:

$$\lim_{n\to\infty} \mathbf{s}[n] = \alpha_1(1)^n\mathbf{v}_1 = \alpha_1\mathbf{v}_1 \tag{11}$$

For systems where the state vector represents fractions or probabilities, its components must sum to 1 ($\mathbf{1}^\top\mathbf{s}[n] = 1$). In this case, $\alpha_1 = \mathbf{1}^\top\mathbf{s}[0] = 1$. The limiting state is simply the eigenvector $\mathbf{v}_1$ corresponding to $\lambda_1 = 1$, normalized to sum to 1. This steady state is also called the **equilibrium state** because once the system reaches it, it no longer changes ($\mathbf{P}\mathbf{s}^* = 1 \cdot \mathbf{s}^* = \mathbf{s}^*$). This vector represents the PageRank scores of the web pages in the network.